

AUDITORÍA — CICLO DE COBRANZA

Logisolve · Calidad de datos para medir DSO (días de cobranza)

Generado: 05/06/2026 07:35 | Fuente: src/app.jsx | Datos seed analizados estáticamente

Auditoría de la calidad de timestamps disponibles para medir Entregado → Facturado → Cobrado. Se analiza el código fuente y los 7 tickets seed. Para datos reales, ejecutar el snippet de la Sección 7.

1. ¿Existe un timestamp confiable para cada estado?

Estado	¿Existe?	Fuente principal	Formato del evento en timeline[]	Campo dedicado
ENTREGADO	✓ Sí (tickets reales) ✗ Parcial (seed)	t.timeline[]	"Estado: Entregado" ← TKT_STATUS (UI) "Entregado" / "Entregado. Xxx" ← seed / texto libre	✗ No existe entregadoAt
FACTURADO	✓ Sí — si se usó el estado intermedio	t.timeline[]	"Estado: Facturado" ← TKT_STATUS Seed data: NINGÚN evento facturado	✗ No existe facturadoAt
COBRADO	✓ Sí — con doble nombre de evento	t.timeline[]	"Estado: Cobrado" ← TKT_STATUS (cambio manual) "Cobrado" ← TKT_COBRADO (botón rápido) ← DISTINTO	✗ No existe cobradoAt

■ Doble nombre del evento cobrado: 'Estado: Cobrado' y 'Cobrado' coexisten. El código analítico ya lo maneja con OR, pero es frágil ante futuros cambios.

■ FUENTE ESTRUCTURADA IGNORADA: t.history[] contiene {type:'status_changed', to:'entregado', ts:'ISO'} — timestamps estructurados sin parsing de texto. El código usa timeline[] en cambio, que requiere búsqueda por substring y es sensible a variaciones de texto.

2. Análisis estático de los 7 tickets seed

Todos tienen status='entregado' y cobrado=false. Ninguno pasó por 'facturado' o 'cobrado'.

ID	Status	Último evento en timeline	Coincide con 'Estado: Entregado'	cobrado	promesaPago
TKT-20260512-001	entregado	"Entregado. Unidad recuperada." 12/05 11:55Z	✗ No (texto libre)	false	26/05/2026
TKT-20260513-001	entregado	"Entregado" 13/05 10:20Z	✗ No (sin 'Estado:')	false	27/05/2026
TKT-20260513-002	entregado	"Entregado" 13/05 13:00Z	✗ No	false	27/05/2026
TKT-20260513-003	entregado	"Entregado" 13/05 12:30Z	✗ No	false	27/05/2026
TKT-20260514-001	entregado	"Entregado" 14/05 09:30Z	✗ No	false	29/05/2026
TKT-20260514-002	entregado	"Entregado" 14/05 12:00Z	✗ No	false	30/05/2026
TKT-20260514-003	entregado	"Entregado" 14/05 22:00Z	✗ No	false	30/05/2026
RESUMEN	7 entregados 0 facturados 0 cobrados	7 tienen timestamp de entrega (via substring 'entregado')	0 de 7 coinciden con 'Estado: Entregado' (búsqueda estricta del código analítico)	0/7	Todos tienen promesaPago

Conclusión seed: ciclo de cobranza completamente incalculable sobre datos seed. Los timestamps de entrega SÍ existen en el timeline pero no coinciden con el patrón de búsqueda 'estado: entregado' usado por el análisis. Ningún ticket tiene facturado ni cobrado.

3. Ciclo completo por ticket — entregado → facturado → cobrado

Datos seed. Para producción real, usar snippet de Sección 7.

ID	Fecha Entregado (timeline)	Fecha Facturado (timeline)	Fecha Cobrado (timeline)	Días Ent →Cob	Días Fac →Cob	Estado
TKT-20260512-001	12/05/2026 11:55	—	—	N/A	N/A	entregado
TKT-20260513-001	13/05/2026 10:20	—	—	N/A	N/A	entregado
TKT-20260513-002	13/05/2026 13:00	—	—	N/A	N/A	entregado
TKT-20260513-003	13/05/2026 12:30	—	—	N/A	N/A	entregado
TKT-20260514-001	14/05/2026 09:30	—	—	N/A	N/A	entregado
TKT-20260514-002	14/05/2026 12:00	—	—	N/A	N/A	entregado
TKT-20260514-003	14/05/2026 22:00	—	—	N/A	N/A	entregado
PROMEDIOS	7/7 tienen timestamp	0/7 tienen timestamp	0/7 tienen timestamp	No calculable	No calculable	

4. Qué se puede y no se puede calcular hoy

Métrica	Calculable?	Condición necesaria	Riesgo / Limitación
Días Entregado → Cobrado	✓ Sí — tickets reales con flujo completo	timeline tiene evento que incluya 'entregado' + evento que incluya 'cobrado' y cobrado > entregado	Seed data excluida (patrón no coincide). Doble nombre del evento cobrado es frágil.
Días Facturado → Cobrado	✓ Sí — si 'facturado' no fue saltado	timeline tiene 'Estado: Facturado' + evento cobrado	Muchos operadores van entregado→cobrado directo, saltando facturado. Muestra sesgada.
Días Entregado → Facturado	✓ Sí — si 'facturado' existe en timeline	Ambos eventos en timeline y facturado > entregado	Incalculable si se salta el estado facturado.
DSO real (días en cartera)	■ Aproximado (promesa ≠ cobro real)	Actualmente usa t.promesaPago como referencia, no la fecha real de cobro	promesaPago es la fecha acordada. El cobro real puede diferir semanas o meses.
Promedio global (todos)	■ Muestra parcial (solo flujo completo)	Solo tickets con AMBOS extremos del intervalo tienen datos para el promedio	El promedio excluye operaciones con estados saltados o sin cobro registrado — sesgo positivo.

5. Problemas exactos — qué eventos o timestamps faltan

ID	Sev.	Problema	Detalle
P-01	ALTA	Doble nombre del evento cobrado	'Estado: Cobrado' (TKT_STATUS) vs 'Cobrado' (TKT_COBRADO). Código usa OR para ambos. Si alguien agrega un tercer path o cambia el texto, el timestamp de cobro se pierde sin error.
P-02	ALTA	Sin campos dedicados de fecha	No existen entregadoAt, facturadoAt, cobradoAt. Todos los timestamps dependen de parsing de texto libre del array timeline[]. Cambios en el texto rompen los cálculos silenciosamente.

P-03	ALTA	Estado 'facturado' frecuentemente omitido	El flujo permite ir de entregado → cobrado directamente. Si el operador usa el botón 'Cobrar' sin pasar por facturado, el intervalo Ent→Fac y Fac→Cob son incalculables. Esto sesga el promedio hacia las operaciones más formales.
P-04	MEDIA	Seed data con texto libre incompatible	7 tickets seed usan 'Entregado. Unidad recuperada.' y 'Entregado' — NO coinciden con 'Estado: Entregado' (búsqueda estricta del código analítico, línea 3075). Todos estos tickets quedan excluidos de los cálculos de velocidad financiera.
P-05	MEDIA	t.history[] ignorado — fuente más robusta disponible	t.history[] contiene {type:'status_changed', to:'entregado', ts:'ISO'} — estructurado, sin texto libre, inmune al problema del doble nombre. El código analítico usa timeline[] y no aprovecha history[].
P-06	BAJA	promesaPago != fecha real de cobro	El DSO aproximado usa t.promesaPago (fecha acordada). La fecha real de cobro solo está disponible como timestamp en el evento 'Cobrado' del timeline.

6. Qué se necesita para medir correctamente el ciclo de cobranza

R-01 (CRÍTICO): Agregar campos dedicados al objeto ticket: entregadoAt, facturadoAt, cobradoAt (ISO strings). Poblarlos automáticamente en TKT_STATUS y TKT_COBRADO. Elimina la dependencia de parsing de texto.

R-02 (CRÍTICO): Unificar el nombre del evento cobrado: elegir UN solo string ('Estado: Cobrado' o 'Cobrado') y aplicarlo en ambos dispatchers. Mientras coexisten dos, el riesgo de regresión es permanente.

R-03 (RECOMENDADO): Migrar el código analítico de timeline[] a history[] para timestamps de estado. history[] ya tiene {type:'status_changed', to:'entregado', ts:'...'} — estructurado y sin texto libre.

R-04 (OPERATIVO): Capacitar a operadores para NO saltar el estado 'facturado' cuando emiten factura. Sin este paso, el intervalo Facturado→Cobrado solo captura operaciones con flujo completo.

7. Snippet de consola — datos reales de producción

Ejecutar en F12 → Consola con la app abierta:

```
const s=JSON.parse(localStorage.getItem('logisolve_state')||'{}');
const ts=(s.tickets||[]).filter(t=>!t._deleted);
// Busca en timeline[] por substring (método actual del código)
const fev=(tl,kw)=>{const e=(tl||[]).find(ev=>(ev.evento||'').toLowerCase().includes(kw));
return e?new Date(e.ts):null;};
// Alternativa robusta: busca en history[] por tipo estructurado
const fevH=(hist,toState)=>{const e=(hist||[]).find(h=>h.type==='status_changed'&&h.to===toState);
return e?new Date(e.ts):null;};
const dias=(a,b)=>a&&b&&b>a?Math.round((b-a)/86400000):null;
const COBR=ts.filter(t=>['entregado','facturado','cobrado','cerrado'].includes(t.status));
console.log('Tickets post-entrega:', COBR.length);
const ciclos=COBR.map(t=>{
const tl=t.timeline||[]; const hist=t.history||[];
// Via timeline (método actual)
const ent=fev(tl,'estado: entregado')||fev(tl,'entregado');
const fac=fev(tl,'estado: facturado');
const cob=fev(tl,'estado: cobrado')||fev(tl,'cobrado');
// Via history (más confiable)
const entH=fevH(hist,'entregado'); const facH=fevH(hist,'facturado');
const cobH=fevH(hist,'cobrado')||hist.find(h=>h.type==='cobrado')?new
Date(hist.find(h=>h.type==='cobrado').ts):null;
return { id:t.id, status:t.status,
ent_tl:ent?.toISOString().slice(0,10), fac_tl:fac?.toISOString().slice(0,10),
cob_tl:cob?.toISOString().slice(0,10), ent_h:entH?.toISOString().slice(0,10),
fac_h:facH?.toISOString().slice(0,10), cob_h:cobH?.toISOString().slice(0,10),
dias_EC_tl:dias(ent,cob), dias_FC_tl:dias(fac,cob), dias_EF_tl:dias(ent,fac),
dias_EC_h:dias(entH,cobH), dias_FC_h:dias(facH,cobH) };
});
```

```
console.table(ciclos);
const avg=arr=>arr.filter(v=>v!=null);
const mean=arr=>arr.length?(arr.reduce((a,b)=>a+b,0)/arr.length).toFixed(1):null;
const ec=ciclos.map(c=>c.dias_EC_tl).filter(v=>v!=null);
const fc=ciclos.map(c=>c.dias_FC_tl).filter(v=>v!=null);
const ef=ciclos.map(c=>c.dias_EF_tl).filter(v=>v!=null);
console.log('Prom Entregado→Cobrado:', mean(ec), 'días | n='+ec.length+'/'+COBR.length);
console.log('Prom Facturado→Cobrado:', mean(fc), 'días | n='+fc.length+'/'+COBR.length);
console.log('Prom Entregado→Facturado:', mean(ef), 'días | n='+ef.length+'/'+COBR.length);
```